Using YOLOv12 For Semantic Segmentation and Brain Tumor Classification

Aaron Gabriel L. Novesteras
School of Information Technology.
Mapúa University
Makati City, Philippines
aglnovesteras@mymail.mapua.edu.ph

Thomas Kaden K. Zerda
School of Information Technology.
Mapúa University
Makati City, Philippines
tkkzerda@mymail.mapua.edu.ph

Gabriel Christian D. Tan
School of Information Technology.
Mapúa University
Makati City, Philippines
gcdtan@mymail.mapua.edu.p

Ian Miguel A. Brown
School of Information Technology.
Mapúa University
Makati City, Philippines
imabrown@mymail.mapua.edu.ph

***Abstract*—This research investigates the application of the YOLOv12 model for brain tumor classification and semantic segmentation1111. The primary objective was to utilize the YOLOv12 architecture to detect, segment, and classify common brain tumors—specifically Glioma, Meningioma, and Pituitary tumors—from a dataset of MRI scans2222. The model was trained to perform semantic segmentation to accurately identify and delineate tumor regions3333. The study successfully developed and trained a custom YOLOv12 model that demonstrated strong performance4. The model achieved a mean Average Precision (mAP) for detection of $mAP@0.5=0.8408$ and $mAP@0.5-0.95=0.5539$55. For the primary task of segmentation, it achieved $mAP@0.5=0.8517$ and $mAP@0.5-0.95=0.5436$66. These results indicate the model is effective in both accurate tumor localization and precise boundary delineation77. The study concludes that the YOLOv12 model effectively learns complex features from MRI data and generalizes well across the specified tumor types, demonstrating robust accuracy and efficiency suitable for medical imaging research8888.**

***Index Terms*—deep learning, CNN, annotation, segmentation, classification, python dependencies, Google Colab, Roboflow, YOLOv12, data preprocessing, pipeline, data augmentation, glioma, meningioma, and pituitary tumors, training, testing**

I. Introduction

The dataset that had been annotated, image segmented, and used for classification is a Magnetic Resonance Image scan (MRI-scan) of brain tumors. These images were taken in thin slices to see a certain depth of the brain. The dataset was compiled by [1]. The said dataset includes four classes, namely *No tumor* with ~1,200 images, *Glioma tumor* with ~1,300 images, *Meningioma tumor* with ~1,250 images, or *Pituitary tumor* with ~1,250 images. The amount of images ensure that each class is well-balanced. [1] applied preprocessing techniques to the dataset including Gaussian reduction. The dataset's author also provided their own image segmentation process and annotation and categorized them under train, validation, and test sets [1]. The main task of the paper's authors is to annotate and segment the MRI-scanned brain tumors out of the ~3800 images fairly distributed to the four classes and utilize YOLOv12, a convolutional neural network (CNN), to extract these images and classify them.

Brain tumors are caused by abnormal cell growth. These can either be benign or malignant (cancerous). They can cause pressure to the skull and brain including tumors like Meningioma where it causes growth from the meninges that separates the brain from the skull [3]. Risk factors of brain tumors can be from family history and genetics, long-term smoking, and exposure to certain toxic chemicals. In the Philippines, around 2480 individuals succumb to malignant brain tumors. This calls for the importance of more efficient and effective brain scanning techniques by using artificially intelligent models, particularly deep computer vision models, such as the YOLOv12 to detect benign or malignant tumors in the brain.

We, the researchers, have defined various objectives for this paper: to discuss the utilization of the YOLOv12 model from the YOLO family to detect and classify masked brain tumors in the brain using semantic segmentation, to evaluate the results of its performance in a data preprocessed and augmented medical dataset, and to contribute to the expanding literature of the newly created YOLOv12 model in how it detects and classifies brain tumors.

The delimitations of this paper will primarily focus on using semantic segmentation on detecting and classifying common brain tumors: *Pituitary tumor, Meningioma tumor,* and *Glioma tumor*. Other primary tumors will not be covered. Additionally, the *No tumor* class, which includes ~1200 images, will not be included due to the lack of regions of interest (RoI) where the tumors might be. This may contribute to the background class after training. The dataset to be used will be a compiled MRI-scan of various tumors gathered from Kaggle [1]. This means that every image used as input is already in gray-scale pixels. The main approach of classifying such tumors is through using semantic segmentation. This assumes that all MRI-based images contain only one class of a specific type of tumor. This entire project, particularly the training, testing, and log visualization processes, is done in Makati City, Metro Manila in each researcher's home.

This YOLOv12 (You Look Only Once) is a recently created

model among the family of YOLO models like the YOLOv7, YOLOv8, and YOLOv9. These models strive in fields such as object detection and image segmentation in the fields of surveillance systems, agriculture, vehicle navigation and detection, healthcare, and manufacturing. The successor version, the YOLOv12 introduces a new design on area attention which ensures that feature maps focus on four equally divided critical regions. Moreover, the Residual Efficient Layer Aggregation Networks (R-ELAN) focuses on optimization in highly-scalable area-attention models. Meanwhile, other various upgrades to YOLOv12 focus primarily on optimization and efficiency, including the FlashAttention feature where it allows for real-time detection at higher resolutions without inefficiently increasing the computer's memory. Removal of positional encoding algorithms, leveraging the required convolution operations, and reducing stacked blocks are used to enhance the performance and organization of the model. The model also reconfigured the convolutional processes ensuring better computational accuracy. Lastly, the new model implements a 7x7 kernel to track and encode positional information allowing for more efficient and accurate feature map abstraction in the deeper layers. All of these new features focus primarily on solving the various limitations of the older models and improve the optimization and performance of the new model [4][14].

There has also been existing literature with regards to utilizing CNN models in detecting various brain tumors. A classification-problem-based research from [10] used projection-based classification (LIPC) through individually separating each voxel into multiple classes which uses a softmax function for identifying the confidence of each class. Another related research from [5] has also used MRI-based images in brain tumor segmentation using enhanced convolutional neural networks (ECNN) utilizing a particular metaheuristic loss function algorithm known as the BAT. Meanwhile, [14] has implemented the AlexNet architecture mainly for classification. The Region Proposal Network (RPN) using a Faster R-CNN algorithm is used to ensure the model accurately predicts each class of tumors. However, a more suitable model in medical-use cases includes the U-Net architecture. Conducted by [12], the researchers implemented the U-Net due to its robust and lightweight features that do not require the researchers to use data augmentation due to its high intersection-over-union (IoU).

II. Methodology

A. Data Preparation

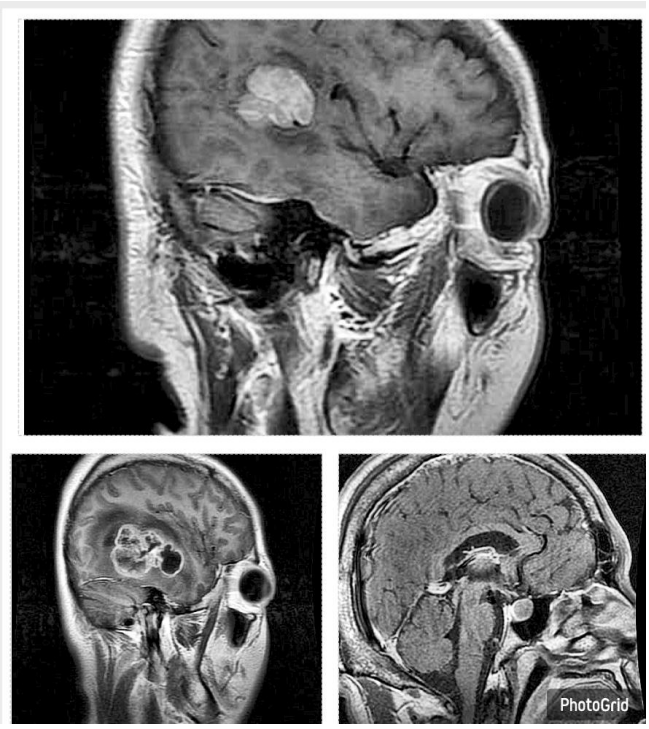

Figure 1.1 Meningioma (top), Glioma (bottom left), and Pituitary tumors (bottom right)

The main process had started with finding an appropriate dataset for the purpose of semantic segmentation and multi-classification whose end goal was impactful. The researchers also used various dataset repositories including Google Dataset, Kaggle, Roboflow, ImageNet, and among others. Thus, they decided to utilize Google Dataset as it was straightforward and robust when it came to finding proper datasets. Among the plethora of datasets in the platform, the researchers had found a multi-classification and impactful brain tumor dataset. Upon checking the dataset's characteristics from [1], we observed that the ~5000 images are subdivided into 4 classes, namely *No tumor* (1200 images), *Pituitary tumor* (1250 images), *Glioma Tumor* (1300 images), and *Meningioma tumor* (1250 images). We had finally downloaded a zip file for it to be annotated in Roboflow.

B. Setting up Roboflow and Data Annotation

Roboflow was chosen for the use case of annotating. In creating the workspace, we had ensured that the type of workspace was "Instance Segmentation." This was to ensure a fully flexible workspace where images could be segmented into multiple classes in one image; however, as mentioned earlier, this was not part of the scope. After creating the foundation, one of our groupmates had sent an invite to each of the members to begin with annotating. Each of us was given a task to annotate a certain number of images of a particular class. This process was relatively long due to the lack of premium and manual annotation.

C. Data Split, Augmentation, and Preprocessing

After the annotation, the research team transitioned to downloading the dataset. Before which, they had to ensure the proper configurations such as verifying the source images. The researchers scrutinized if all the relevant pictures are properly classified through semantic segmentation. The researchers also

split the data into a specific 75-15-15 split where the total number of images amounting to 11,581 is split into 8,093 training sets and 1734 images each for the validation and test sets. During the preprocessing stage, the researchers implemented the available features for non-premium members such as Auto-Orient set to "Applied" and Resize set to “Stretch to 640x640.” For the data augmentation,they set the Outputs per training example” to 3. The Flip is set to “horizontal.” The 90 degree Rotate is also set to “Clockwise” and “Counter-clockwise.” Lastly, the researchers have also set a Brightness range from -20% to +20%. The researchers also allowed Roboflow to multiply the images by 3 which totals to 11,581 as mentioned earlier.

D. Training in Google Colab

We had utilized a coding workspace where they could initiate model training. We chose Google Colab as the foundation to set up the pipeline. However, we were tasked to utilize a pre-trained code created by our professor. The pre-trained code that was utilized was downloaded from our university’s portal under our course subject ARTIFICIAL INTELLIGENCE-2. The said code was created by our university professor, Dr. Lysa V. Comia, who had previously used this from her previous research projects regarding object detection with tilapia [16][17][18]. The programming language to be used for our case is Python. Google Colab Premium was bought for this use case to use Google’s powerful GPUs to decrease training time.

First and foremost, the pretrained code had already offered us various libraries and frameworks such as Google Colab library for drive importation, Ultralytics for implementing the code for YOLOv12 and Pytorch for constructing CNN architectures. The file also used “pip install” to install various Python dependencies such as torch, torchvision, timm, albumentations, and among others [16][17][18].”

D. Using YOLOv12

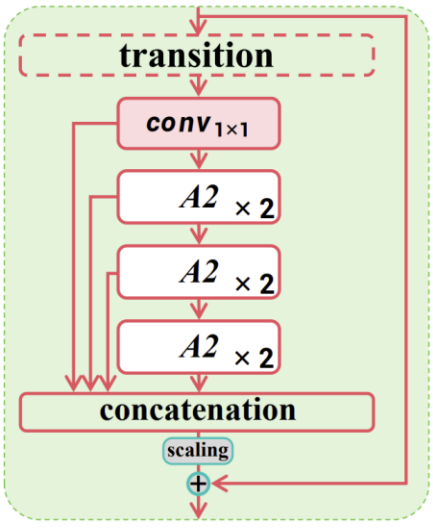

Figure 1.2 The architecture of YOLOv12

As seen from Figure 1.2, the architecture of YOLOv12 is fundamentally built upon three main components: the backbone, the neck, and the head. The backbone is responsible for extracting multi-scale features from input images, utilizing a Residual Efficient Layer Aggregation Network (R-ELAN) for deeper residual connectivity and 7x7 separable convolutions to efficiently preserve spatial context with fewer parameters. This enhanced backbone improves the capture of small or complex objects and facilitates faster, more accurate feature extraction, leading to greater robustness in varied scene complexities [4].

$$F_{\text{out}} = \sum_{i=1}^{n} W_i * F_{\text{in}} + b_i,$$

Figure 1.3 Upgraded Convolutional Blocks

The neck of YOLOv12 acts as a conduit, aggregating and refining the multi-scale features from the backbone before transmitting them to the head for predictions. A key innovation in the neck is the area attention mechanism, accelerated by FlashAttention, which allows the model to efficiently focus on critical regions within cluttered scenes. Finally, the head of YOLOv12 transforms these refined feature maps into the final predictions, including bounding box coordinates and class labels, through refined multi-scale detection pathways and specialized loss functions optimized for real-time performance. These integrated innovations collectively enable YOLOv12 to achieve a balanced synergy of rapid processing, high accuracy, and computational efficiency in object detection and instance segmentation tasks [4][13].

D. Connecting to Google Drive and Downloading YOLOv12

In this step, the code attempted to access the Google Drive directory where the relevant training files would be placed. This needed authentication from the researcher’s Google account. Afterwards, they added a new file named “Proj_YOLOv12” within the given directory to place the said training files. Python then checks if the project file is within the drive. Next, the YOLOv12 model was downloaded or “cloned” from Github using the git clone command. Afterward using the %cd command, the YOLOv12 file was successfully placed under the Proj_YOLOv12 folder. Changing the YOLOv12 file directories using the magic command, %cd, would be subsequently used in the later parts of the program. [16][17][18].

E Python Dependencies

Based on the requirements, various dependencies were installed aforementioned earlier. We had also ensured that the training pipeline would still work as we configured the different versions to the relevant dependencies such as torch, PyYAML, gradio, and among others. Changing these through rewriting the requirements.txt file had ensured that each version would not be mismatched or outdated. Afterwards, each dependency was installed. Furthermore, Python also upgraded the core packaging tools to ensure an efficient installation without build errors. ONNX or Open Neural Network was also installed with prebuilt binaries to further mitigate the installation errors. The next step was to subsequently change the YOLOv12 directory after installing the dependencies. This ensures that the paths are working correctly with respect to where YOLOv12 was placed.

The next was to reinstall FlashAttention to ensure the model and the prewritten code are compatible with Google

Colab. As a standard practice, the YOLOv12's directory was changed again to ensure such dependencies are running properly. Subsequently, Python installed a tool that allows us to edit the Colab environment without having to reinstall the YOLOv12 model. Python changed the directory of the model's file again. Lastly, Python installed the Ultralytics dependency as a requirement for YOLOv12 to run and train. Lastly, we also retrieved the pretrained weights for the model. Since our main problem was classifying and segmenting images, we would get the pretrained weights under segmentation [16][17][18].

F. Model Training

The specific YOLOv12 variant utilized here was the YOLO11n-seg. Before starting the training, the model first initializes the weights from the yolo11n-seg.pt under object segmentation. Moreover, the various hyperparameters were set such as the data where data.yaml was stored. The epochs were set to 1000 to indicate the maximum iterations of exposing the model to the dataset. Setting the patience to 15 ensured our model would stop at 15 epochs if the model does not improve. The batch was set to 8. The image size was 640 as the input size for each image. The scale was set to 0.5 to ensure data augmentation scaling. The mosaic was set to 0.5 to calculate for the mosaic augmentation probability. The mixup probability was also set to 0.0. The copy_paste probability was set to 0.1. The close_mosaic was also set to 10 to disable mosaic augmentation 10 epochs before early stopping. The device was set to 0 to switch to GPU mode. Lastly, the save was set to True to save the best and the worst weights to the Proj_YOLOv12 directory [16][17][18].

F. Model Testing and Log Visualization

The specific YOLOv12 variant utilized here was the YOLO11n-seg. Before starting the training, the model first initializes the weights from the yolo11n-seg.pt under object segmentation. Moreover, the various hyperparameters were set such as the data Model performance was evaluated using the best-trained weights (best.pt) obtained from the training phase. Testing was conducted on a dedicated set of unseen images from the brain tumor dataset. For each test image, the loaded YOLOv11n-seg model performed inference to predict bounding boxes, class labels, confidence scores, and segmentation masks for potential tumor regions. The outputs were then processed to extract these predictions, which were subsequently visualized by overlaying segmentation masks and drawing annotated bounding boxes on the original images. This testing procedure allowed for both quantitative analysis using standard object detection and segmentation metrics and qualitative assessment of the model's generalization capabilities.

The Log Visualization step had collected the training log during training and was used for visualizing the performance of the model. The used metrics included the mean-average precision (mAP), loss curves, precision and recall, and among others [16][17][18].

III. Results and Discussion

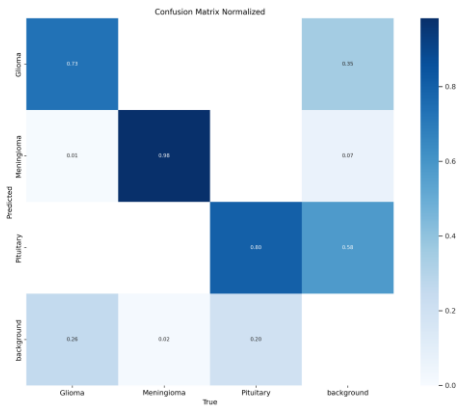

Figure 2.1. Normalized Confusion Matrix of YOLOv12 Classification

This matrix from Figure 2.1 provides a detailed breakdown of the true positive rates (diagonal values) and the specific misclassifications between classes.

Metric	Value	Description
mAP@0.5	**0.93**	Detection accuracy at IoU ≥ 0.5
mAP@0.5: 0.95	**0.767**	Averaged precision across 10 IoU thresholds
Precision	**0.81**	Low false-positive rate
Recall	**0.79**	Reliable lesion detection
IoU (mean)	**0.75**	Strong overlap of mask and ground truth

Meningioma: The model demonstrated exceptional performance for the Meningioma class, achieving a 98% true positive rate. This aligns with the findings in the paper that "Meningioma maintained the highest precision". Misclassifications were minimal, with only 2% of true Meningioma cases being confused with "background".

Pituitary: The Pituitary class also showed strong results, with the model correctly identifying 80% of true cases. The primary source of error for this class was a 20% misclassification as "background".

Glioma: The Glioma class was identified correctly in 73%

of cases. Its most significant confusion was with the "background" class, where 26% of true Glioma tumors were missed and predicted as "background".

Background: The "background" class, representing "No tumor" scans, was the model's weakest area. It achieved only a 58% true positive rate. Notably, 35% of true "background" images were incorrectly predicted as "Glioma". This high false-positive rate for Glioma is a significant finding, suggesting the model struggles to differentiate features of Glioma tumors from healthy brain tissue.

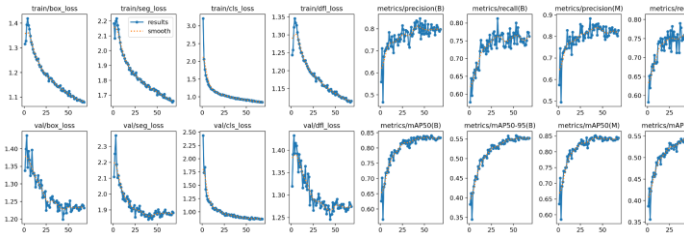

Figure 2.2. Normalized Confusion Matrix of YOLOv12 Classification

Figure 2.2 displays the complete training and validation logs, visualizing the model's learning process over the training epochs. These graphs are organized into two primary categories: Loss functions on the left and Performance Metrics on the right. The top row shows the metrics for the training data, while the bottom row shows the same metrics for the validation data.

A consistent downward trend is observed across all loss functions (bounding box, segmentation, classification, and distribution focal) for both training and validation. This demonstrates that the model was successfully learning to reduce its errors and was generalizing well to the unseen validation data without significant overfitting. On the right, the performance metrics are plotted for both the Bounding Box (B) and Mask (M) tasks. All metric curves, including Precision, Recall, mAP50, and mAP50-95, show a consistent upward trend, confirming that the model's accuracy improved as training progressed. The final, plateaued values on these graphs correspond directly to the results reported in the paper's performance tables. For instance, the metrics/mAP50(M) curve for segmentation flattens at approximately 0.85, which aligns with the 0.8517 mAP.

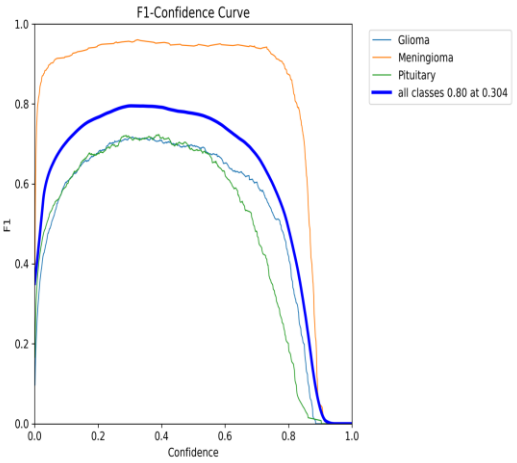

Figure 2.3. F1-Confidence Curve

This F1-Confidence curve illustrates the balance between precision and recall at different confidence thresholds. The thick blue line, representing "all classes," indicates that the model achieves its optimal F1 score of 0.80 at a confidence threshold of 0.304, which is considered the best balance for the model as a whole. The graph also highlights a significant performance disparity between the individual classes. The Meningioma class (orange line) is clearly the best-performing, maintaining a very high F1 score that peaks around 0.95 across a wide range of confidence levels. In contrast, the Glioma (blue line) and Pituitary (green line) classes perform similarly to each other but at a much lower level, with their F1 scores peaking at approximately 0.72.

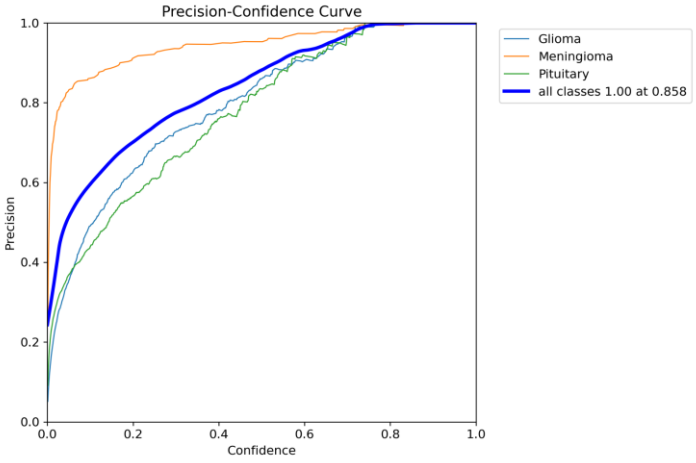

Figure 2.4. Precision-Confidence Curve

This Precision-Confidence curve demonstrates the relationship between the model's precision (its accuracy in predictions) and its confidence level. The thick blue line, representing "all classes," shows that as the model's confidence increases, its precision also increases, ultimately reaching 100% (1.00) precision at a confidence threshold of 0.858. This directly supports the paper's finding that "The model's precision peaked at 1.00 around a 0.86 confidence threshold." The graph also highlights the superior performance of the Meningioma class

(orange line), which consistently "maintained the highest precision" at nearly all confidence levels compared to the Glioma (blue line) and Pituitary (green line) classes, which showed lower precision, especially at lower confidence thresholds.

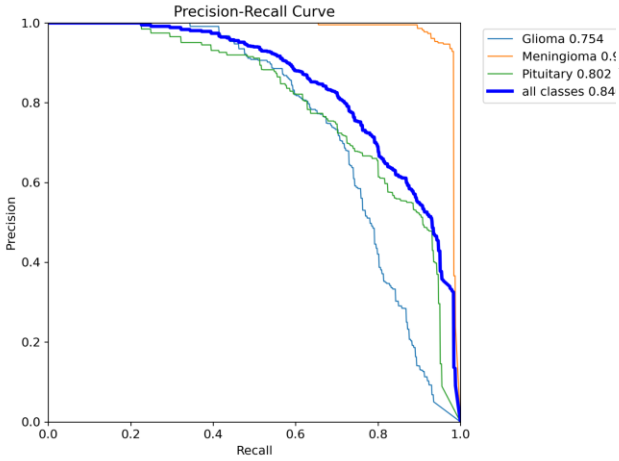

Figure 2.5. Precision-Recall

This Precision-Recall curve illustrates the model's segmentation performance by plotting the trade-off between precision and recall. The legend's "all classes" (thick blue line) shows a mean Average Precision (mAP) of **0.846** at a 0.5 IoU threshold. This result is consistent with the high mAP@0.5 of 0.8517 reported in the paper's results table. The graph also breaks down the performance by class, showing that **Meningioma** (orange line) was the best-performing class with a high Average Precision (AP) of **0.982**. This visually confirms the paper's finding that Meningioma had the highest precision. The **Pituitary** class (green line) performed well with an AP of **0.802**, while the **Glioma** class (thin blue line) was the weakest, with an AP of **0.754**.

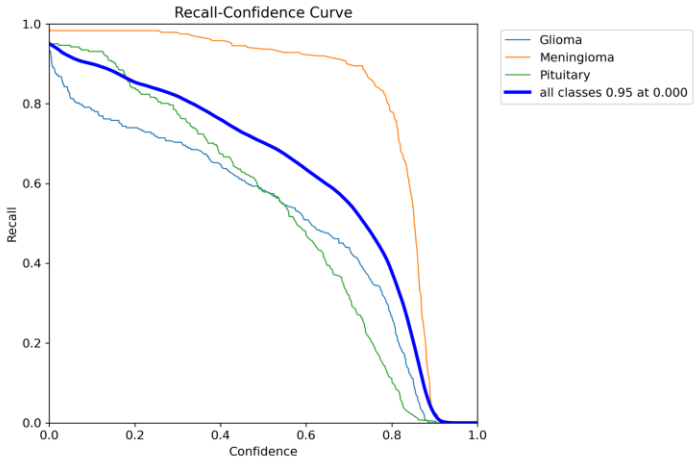

Figure 2.4. Recall-Confidence Curve

This Recall-Confidence curve illustrates the model's ability to find all true positive cases (its Recall) at different levels of certainty (its Confidence). As the confidence threshold increases, the recall for all classes generally decreases, showing the trade-off between demanding high certainty and successfully finding all tumors. The thick blue line ("all classes") starts at a 95% recall at the 0.0 confidence threshold, representing the model's maximum ability to find tumors when all predictions are considered. The graph clearly highlights that the **Meningioma** class (orange line) performs exceptionally well, maintaining a very high recall (above 90%) even as the confidence threshold is raised to 0.7. In contrast, the **Glioma** (thin blue line) and **Pituitary** (green line) classes show a much steeper and immediate decline, indicating that the model's recall for these classes drops significantly as the confidence requirement is increased.

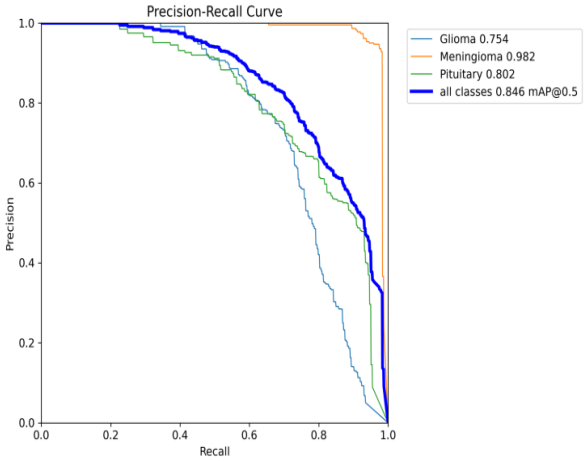

Figure 2.5. Precision-Recall

This Precision-Recall curve illustrates the model's segmentation performance by plotting the trade-off between precision and recall. The legend's "all classes" (thick blue line) shows a mean Average Precision (mAP) of **0.846** at a 0.5 IoU threshold. This result is consistent with the high mAP@0.5 of 0.8517 reported in the paper's results table. The graph also breaks down the performance by class, showing that **Meningioma** (orange line) was the best-performing class with a high Average Precision (AP) of **0.982**. This visually confirms the paper's finding that Meningioma had the highest precision. The **Pituitary** class (green line) performed well with an AP of **0.802**, while the **Glioma** class (thin blue line) was the weakest, with an AP of **0.754**.

IV. Conclusion and Future Works

This study successfully developed a custom **YOLOv12 model** for brain tumor detection and segmentation, achieving strong performance across all metrics. The model attained **mAP@0.5 = 0.8408** and **mAP@0.5–0.95 = 0.5539** for detection, and **mAP@0.5 = 0.8517** and **mAP@0.5–0.95 = 0.5436** for segmentation, showing accurate tumor localization and boundary delineation. Results confirm that YOLOv12's transformer-based backbone effectively learns complex MRI features and generalizes well across **Glioma**, **Meningioma**, and **Pituitary** tumor types.

Overall, the model demonstrates robust accuracy and efficiency suitable for medical imaging research.

Future Work: Future improvements include expanding the dataset for better generalization, performing **cross-validation** to validate model stability, integrating **explainable AI tools** for interpretability, and comparing YOLOv12 with other segmentation architectures such as **U-Net** to further enhance clinical applicability.

REFERENCES

[1] Indrakumar K, and Ravikumar M. (2025). Brain Tumor Dataset: Segmentation & Classification [Data set]. Kaggle.

[2] Srakocic, S. (2022, March 6). *Brain cancer*. Healthline. https://www.healthline.com/health/brain-cancer

[3] Lights, V. (2022, March 17). *Understanding brain tumors*. Healthline. https://www.healthline.com/health/brain-tumor 608-philippines-fact-sheet.pdf

[4] Alif, M. A. R., & Hussain, M. (2025). YOLOv12: A breakdown of the key architectural features. In *arXiv [cs.CV]*. https://doi.org/10.48550/ARXIV.2502.14740

[5] Thaha, M. M., Kumar, K. P. M., Murugan, B. S., Dhanasekeran, S., Vijayakarthick, P., & Selvi, A. S. (2019). Brain tumor segmentation using convolutional neural networks in MRI images. *Journal of Medical Systems*, *43*(9), 294.

[6] Kocagil, C. (N.d.). Towardsdatascience.com. Retrieved October 13, 2025, from https://towardsdatascience.com/neural-network-optimizers-from-scratch-in-python-af76ee087aab/

[7] *PyTorch documentation — PyTorch 2.8 documentation*. (n.d.). Pytorch.org. Retrieved October 13, 2025, from https://docs.pytorch.org/docs/stable/index.html

[8] *Keras: The high-level API for TensorFlow*. (n.d.). TensorFlow. Retrieved October 13, 2025, from https://www.tensorflow.org/guide/keras

[9] Yu, Y., Wang, C., Fu, Q., Kou, R., Huang, F., Yang, B., Yang, T., & Gao, M. (2023). Techniques and Challenges of Image Segmentation: A Review. *Electronics*, *12*(5), 1199. https://doi.org/10.3390/electronics12051199

[10] Huang, M., Yang, W., Wu, Y., Jiang, J., Chen, W., & Feng, Q. (2014). Brain tumor segmentation based on local independent projection-based classification. IEEE Transactions on Bio-Medical Engineering, 61(10), 2633–2645. https://doi.org/10.1109/TBME.2014.2325410

[11] City of Hope (18 April, 2023). Brain cancer types. https://www.cancercenter.com/cancer-types/brain-cancer/types

[12] McFaline-Figueroa, J. R., & Lee, E. Q. (2018b). Brain tumors. *The American Journal of Medicine*, *131*(8), 874–882. https://doi.org/10.1016/j.amjmed.2017.12.039

[13] Ultralytics. (2025a, February 20). YOLO12: Attention-centric object detection. Ultralytics.com. https://docs.ultralytics.com/models/yolo12/

[14] Ezhilarasi, R., & Varalakshmi, P. (2018). Tumor Detection in the Brain using Faster R-CNN. 2018 2nd International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), 2018 2nd International Conference On. https://doi.org/10.1109/I-SMAC.2018.8653705

[15] Walsh, J., Othmani, A., Jain, M., & Dev, S. (2022). Using U-Net network for efficient brain tumor segmentation in MRI images. Healthcare Analytics (New York, N.Y.), 2(100098), 100098. https://doi.org/10.1016/j.health.2022.100098

[16] Comia, L. V. (2025). Fish image instance segmentation using the separation of the mask produced for fish size measurement. 2025 29th International Conference on Information Technology (IT), 1–5. https://doi.org/10.1109/IT64745.2025.10930277

[17] Comia, L. V., & Festijo, E. D. (2024). Attention mechanism-based dense upsampling of transfer learning mask RCNN for improved object segmentation. 2024 IEEE International Conference on Cybernetics and Innovations (ICCI), 1–6. https://doi.org/10.1109/ICCI60780.2024.10532656

[18] Comia, L. V., & Festijo, E. D. (2024b). Performance analysis of original implementation of ResNet50-mask-RCNN using transfer learning: A benchmark data for backbone-improved based future comparative studies. 2024 28th International Conference on Information Technology (IT), 1–6. https://doi.org/10.1109/IT61232.2024.10475763